Reducing Off-Chip Prefetch Request Latency of LLC Hardware Prefetchers via Neural Prediction

Zhengwei Huang
College of Computer Science and Technology, National University of Defense Technology
Changsha, Hunan, China
Key Laboratory of Advanced Microprocessor Chips and Systems
Changsha, Hunan, China
huangzhengwei@nudt.edu.cn

Wei Guo
College of Computer Science and Technology, National University of Defense Technology
Changsha, Hunan, China
Key Laboratory of Advanced Microprocessor Chips and Systems
Changsha, Hunan, China
wineer_guowei@nudt.edu.cn

Yongwen Wang
College of Computer Science and Technology, National University of Defense Technology
Changsha, Hunan, China
Key Laboratory of Advanced Microprocessor Chips and Systems
Changsha, Hunan, China
yongwen@nudt.edu.cn

Abstract

LLC hardware prefetchers are a pivotal technique in modern high-performance processors for hiding memory access long latency. However, even accurate prefetch requests generated by LLC prefetchers experience long latency when they must traverse the LLC before accessing off-chip main memory.

To reduce latency of the off-chip prefetch request generated by LLC prefetchers, we propose Artemis, an off-chip prefetch prediction approach. Artemis employs a neural predictor with confidence learning mechanism to predict whether a prefetch request generated by LLC prefetchers will miss in the LLC. When a miss is predicted, Artemis issues a speculative prefetch request directly to the main memory controller, while the regular prefetch request concurrently accesses the LLC. If Artemis's prediction is correct, the regular prefetch request eventually misses in the LLC and waits for the speculative request to return the data fetched from main memory. By doing so, Artemis hides the LLC access latency from the critical path of off-chip prefetch requests.

Our extensive evaluation on diverse workloads demonstrates that Artemis provides consistent performance improvements across a wide range of system configurations, including varying core counts, state-of-the-art hardware prefetchers, LLC access latencies, and main memory bandwidths, while incurring only 9.38 KB of storage overhead.

CCS Concepts

• **Computer systems organization** → **Multicore architectures**.

Keywords

Architecture, Data Prefetching, Cache Miss Prediction, Memory Hierachy, Processor Performance

ACM Reference Format:

Zhengwei Huang, Wei Guo, and Yongwen Wang. 2026. Reducing Off-Chip Prefetch Request Latency of LLC Hardware Prefetchers via Neural Prediction. In *38th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA '26), July 06–10, 2026, London, United Kingdom.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3816782.3819187

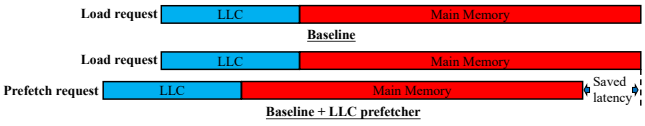

Figure 1: Illustration of latency savings for load requests achieved by LLC prefetchers.

SPAA '26, London, United Kingdom

ACM ISBN 979-8-4007-2761-0/2026/07
https://doi.org/10.1145/3816782.3819187

1 Introduction

In the past few decades, long-latency memory accesses have been one of the major factors limiting processor performance improvement [3, 6–8, 15, 24, 27, 29–32, 38, 52]. Hardware data prefetchers [1–3, 7, 16, 17, 25, 39, 44, 45, 51] are an effective technique to reduce such long latency in modern high-performance processors. LLC hardware prefetchers [1, 3, 7, 44], an important class of hardware data prefetchers, predict future accesses and generate prefetch requests by analyzing the L2 miss stream in the physical address space. These prefetch requests first access the LLC; upon a miss, they proceed to access off-chip main memory and fill the prefetched data into the LLC. By issuing prefetch requests ahead of corresponding load requests, LLC prefetchers can effectively hide part of the memory access latency of load requests (as illustrated in Fig. 1).

Although LLC prefetchers effectively improve processor performance, we observe two key trends in new processor designs that present significant opportunities for further improving processor performance. First, a large number of accurate but late prefetch requests generated by LLC prefetchers miss in the LLC and access off-chip main memory when the processor runs various memory-intensive workloads. We collect these data (shown in Figure 3) for state-of-the-art LLC prefetchers, including Bingo, Pythia, SPP-PPF, and RICH, using the industry-standard ChampSim simulator. In modern out-of-order cores, where a single LLC miss incurs hundreds of stall cycles, the Misses Per Kilo Instructions (MPKI) levels observed in Figure 3 translate to thousands of wasted execution cycles per 1k instructions. This constitutes a severe performance bottleneck. Second, an increasing fraction of the latency of accurate off-chip prefetch requests generated by LLC prefetchers is spent accessing the LLC due to the increasing size of the LLC [34, 35, 47].

The goal in this work is to reduce the latency of accurate off-chip prefetch requests generated by the LLC data prefetcher by removing LLC access latency from the critical path of these requests. To this end, we propose Artemis, an off-chip prefetch prediction approach. The key novelties of Artemis are as follows: (1) accurately predicting whether a prefetch request will miss in the LLC and access off-chip main memory, and (2) upon predicting a miss, issuing a speculative prefetch request (called an Artemis request) with the same physical address as the regular prefetch request directly to the main memory controller, while the regular prefetch request concurrently accesses the LLC. If Artemis's prediction is correct, the regular prefetch request eventually misses in the LLC and waits for the Artemis request to return the data fetched from main memory. If the prediction is incorrect, the prefetch request hits in the LLC, and the Artemis request is dropped. By doing so, Artemis hides the LLC access latency of off-chip prefetch requests (as illustrated in Fig. 2) when predictions are correct, while keeping the full coherence of the on-chip cache hierarchy when predictions are incorrect.

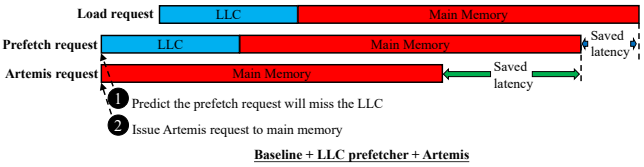

Figure 2: Illustration of latency savings for off-chip prefetch requests achieved by Artemis.

To enable Artemis, we propose OPP, a novel and lightweight off-chip prefetch predictor based on a perceptron neural model [6, 9, 18, 23, 26, 42, 44], incorporating confidence learning mechanism. OPP uses multiple features (e.g., cacheline offset, byte offset) of a prefetch request generated by LLC prefetchers to predict whether it will access off-chip main memory. The key novelties of OPP are summarized as follows. First, OPP is the first off-chip prefetch predictor designed based on the perceptron neural model. Second, we propose six new features tailored for off-chip prefetch prediction, enabling OPP to comprehensively capture the memory access patterns of the program. Third, we introduce confidences that scale the impact of each feature on the prediction outcome. By dynamically adjusting these confidence values during training, OPP adaptively reinforces or attenuates feature contributions across different program phases, thereby enhancing overall prediction performance.

The key contributions of this paper are outlined below:

- We identify two key opportunities to improve modern processor performance. First, a large number of accurate but late prefetch requests generated by LLC prefetchers miss in the LLC and access off-chip main memory when modern processors run various memory-intensive workloads. Second, an increasing fraction of the latency of accurate off-chip prefetch requests is spent accessing the LLC due to the increasing size of the LLC.
- We introduce Artemis, a novel off-chip prefetch prediction technique that hides the LLC access latency of off-chip prefetch requests generated by LLC prefetchers.
- We propose OPP, a novel off-chip prefetch predictor based on a perceptron neural model, incorporating a confidence learning mechanism. OPP uses multiple features of a prefetch request to accurately predict whether it will access off-chip main memory.
- We show that Artemis significantly improves processor performance across various benchmark suites and processor system configurations, including different core counts, LLC prefetchers, LLC latencies, and main memory bandwidths.

2 Motivation

A large number of accurate but late prefetch requests generated by LLC prefetchers access the off-chip main memory. We evaluate state-of-the-art prefetchers configured as LLC prefetchers using 80 workload traces from the SPEC CPU 2006 [13], SPEC CPU 2017 [14], Ligra [40], CVP [33], and GAP [4] benchmark suites. Figure 3 presents the average LLC MPKI incurred by accurate but late prefetches across these benchmark suites.

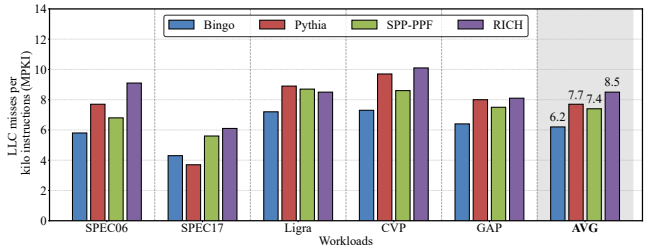

Figure 3: Average LLC MPKI incurred by accurate but late prefetches.

We make two key observations from Figure 3. First, the LLC MPKI incurred by accurate but late prefetch requests exhibits significant variation across different LLC prefetchers and benchmark suites. Second, across 80 workload traces, the prefetch requests generated by Bingo, Pythia, SPP-PPF, and RICH incur an average LLC MPKI of 6.2, 7.7, 7.4, and 8.5, respectively. We conclude that while state-of-the-art LLC prefetchers effectively improve processor performance, the long latency of accurate but late off-chip prefetch requests generated by them presents a significant opportunity for further performance improvement.

An increasing fraction of the latency of accurate off-chip prefetch requests is spent accessing the LLC. We observe that, to accommodate workloads with large data footprints, the LLC in recent processors has increased significantly in size and design complexity [5, 20, 46]. On the one hand, a larger LLC can improve processor performance by preventing more load requests from accessing off-chip main memory. On the other hand, as LLC sizes increase and the complexity of their designs and on-chip networks grows, access latencies of LLC continue to increase [6]. An analysis of the Intel Alder Lake processor core reveals that LLC access latencies have increased to 17 ns [35], corresponding to 68 cycles for a core running at 4 GHz.

3 Artemis: The Goal and Key Novelties

This work aims to improve processor performance by removing LLC access latencies from the critical path of accurate off-chip prefetch requests generated by LLC prefetchers.

3.1 Potential Benefits

To quantify the potential benefit of Artemis, we model a processor system equipped with an Ideal Artemis mechanism in our simulation framework. In this system, Artemis accurately predicts whether each accurate prefetch request generated by LLC prefetchers will miss in the LLC, thereby removing the LLC access latencies for accurate but late off-chip prefetch requests. Figure 4 shows the performance improvement provided by the combination of LLC prefetchers and Ideal Artemis for a single-core processor, normalized to a no-prefetching baseline system. We observe from Figure 4 that Ideal Artemis consistently provides significant performance improvement when combined with each of the four high-performance LLC prefetchers. Specifically, Ideal Artemis improves performance by 8.2%, 9.0%, 8.7%, and 9.9% on top of Bingo [3], Pythia [7], SPP-PPF [9], and RICH [1], respectively.

Based on these results, we conclude that Artemis provides significant performance potential when combined with various high-performance LLC prefetchers.

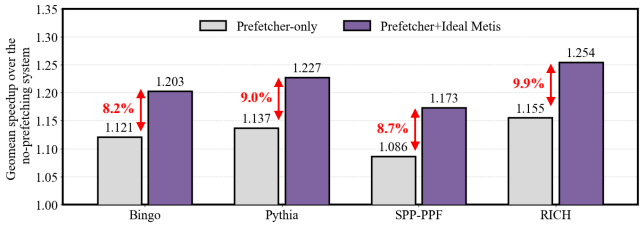

Figure 4: Single-core performance improvement provided by the combination of state-of-the-art LLC prefetchers and Ideal Artemis.

4 Artemis: Design Overview

Figure 5 shows a high-level overview of Artemis. As the key component of Artemis, OPP is designed to make highly accurate off-chip prefetch predictions. For each prefetch request generated by LLC prefetchers, OPP predicts whether or not the prefetch request will go off-chip and access main memory ❶. If the prefetch request is predicted to go off-chip, Artemis issues a speculative prefetch request (called an Artemis request) with the same physical address as the regular prefetch request directly to the main memory controller, while the regular prefetch request concurrently accesses the LLC ❷. If the prediction is correct, the regular prefetch request eventually misses in the LLC and waits for the ongoing Artemis request to return the corresponding data fetched from main memory, thereby removing the LLC access latency from the critical path of the correctly-predicted off-chip prefetch request ❸. If an Artemis request returns from main memory in the absence of a matching regular prefetch request, Artemis discards the fetched data and precludes its insertion into the on-chip cache hierarchy. By doing so, Artemis maintains the coherence of the on-chip cache hierarchy, even in the event of mispredictions. Upon completion of each regular prefetch request, Artemis trains OPP based on whether the request actually accesses off-chip main memory ❹.

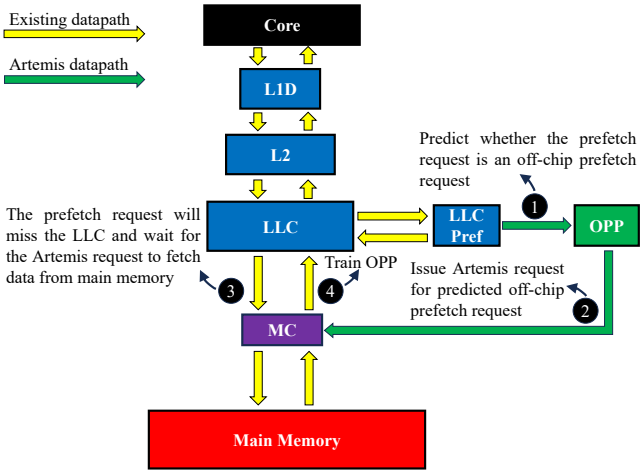

Figure 5: Overview of Artemis.

5 Artemis: Detailed Design

We first detail the design of OPP in Section 5.1, and then discuss the changes Artemis introduces to the on-chip cache access datapath in Section 5.2.

5.1 OPP Design

We design OPP based on a perceptron neural model, incorporating a confidence learning mechanism. We choose the perceptron neural model for the following three key reasons. First, the perceptron can provide superior prediction performance for the processor through multi-feature online learning. Second, the perceptron can be implemented with low storage overhead. Third, the perceptron has been widely used in various processor prediction components such as branch predictors, prefetch filters, reuse predictors, and off-chip load predictors, and experimental results show that designing prediction components using perceptrons can significantly improve processor performance. We choose to augment the perceptron neural model with a confidence learning mechanism for the key reason that the predictive reliability of individual features fluctuates across program phases. The confidence learning mechanism allows the prediction model to dynamically scale feature reliability, adaptively reinforcing the contribution of effective features while attenuating noise to maximize the prediction performance of the model.

OPP Overview. Figure 6 shows an overview of OPP. We use a hashed perceptron [6, 23] to implement the perceptron neural model. The hashed perceptron mainly comprises hashing units for generating indices into weight tables and weight tables for storing feature weights. Although the hashed perceptron has already been applied to various microarchitectural prediction components, this work is the first to apply it to predicting whether prefetch requests generated by LLC prefetchers will miss in the LLC. Unlike prior

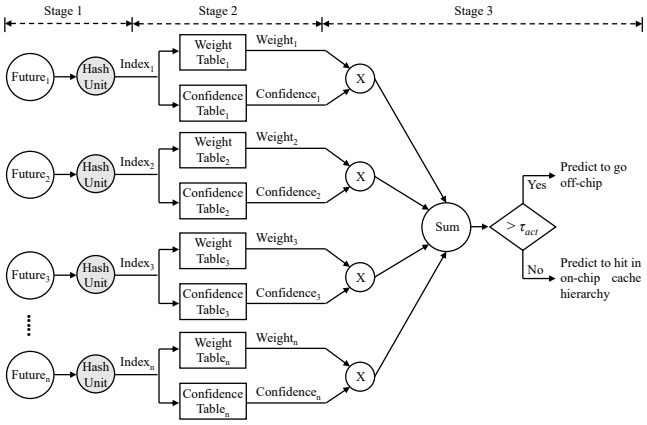

Figure 6: Overview of OPP.

designs based on hashed perceptrons, we introduce a dynamically adjustable confidence to the output of each weight table, scaling the contribution of the corresponding feature to the final prediction. The confidence learning mechanism updates the confidence of each feature based on its accuracy: it increments the confidence if the prediction matches the actual outcome, and decrements it if the prediction contradicts the actual outcome.

5.1.1 Making a Prediction. For prefetch requests generated by LLC prefetchers (Step 1 in Figure 5), OPP predicts whether the prefetch request will go off-chip and access main memory. The prediction procedure of OPP consists of three stages, as shown in Figure 6. In the first stage, OPP extracts a set of features from the current prefetch request and request history (Table 1 shows the list of features used by OPP). Subsequently, hashing units compute hash values for the set of features. In the second stage, the generated hashes index the weight and confidence tables to retrieve 5-bit signed weights (range: [−16, 15]) and 4-bit unsigned confidence values (range: [0, 15]), respectively. In the third stage, each feature weight is scaled by a confidence value to reflect its current predictive reliability. The scaled values are accumulated to generate the total prediction score. If the score exceeds the activation threshold (τ_{act}), OPP predicts that the prefetch request will go off-chip; otherwise, OPP predicts that the prefetch request will hit in the on-chip cache hierarchy. For each prefetch request, the hash of each feature, along with the sum of all feature weights, is stored in the corresponding prefetch queue (PQ) entry, so that they can be reused to train OPP when the prefetch request returns (step 4 in Figure 5). Notably, sharing prediction logic between the LLC prefetcher and Artemis is infeasible. Their divergent training streams (L2 misses vs. past prefetch outcomes) and targets (address generation vs. miss prediction) demand independent parameters; fusing them would cause destructive parameter conflict and degrade predicion performance.

The reasons for our choice of 5-bit signed weight values and 4-bit unsigned confidence values are as follows. First, the use of 5-bit signed weight values provides the necessary dynamic range to determine the prediction direction (hit or miss) and quantify the correlation between program features and the target outcome. Second, we use unsigned confidence values because they are employed to enhance or attenuate the influence of features on the final prediction outcome. Third, 5-bit signed weight values and 4-bit unsigned confidence values provide sufficient prediction performance for the OPP. Further increasing these bit-widths yields only marginal prediction performance gains, while simultaneously incurring unnecessary hardware overhead.

5.1.2 Training the OPP. Artemis trains the weight and confidence of the OPP predictor once the prefetch request returns (step 4 in Figure 5). The weight training process consists of two stages. In the first stage, the hashed values (i.e., weight table indices) and the sum of all feature weights are retrieved from the PQ entry. If the sum of all feature weights lies between the negative saturation threshold T_{NE} and the positive saturation threshold T_{PO}, weight training is triggered. This saturation check prevents the weight values of individual features from becoming overly saturated, enabling OPP to quickly adapt its learning to changes in program phases. In the second stage, if weight training is triggered, Artemis retrieves the weights from the weight tables using the hashed value of each feature and updates the weights. If the prefetch request actually went off-chip, the weight of each feature is incremented by one; otherwise, the weight of each feature is decremented by one. This simple weight update mechanism adjusts each feature weight toward the true outcome, gradually improving prediction performance.

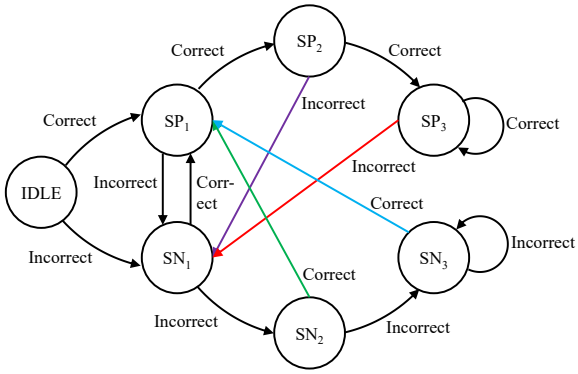

Figure 7: State transitions of the FSM.

We employ a confidence learning mechanism to train the confidence of each feature based on its prediction accuracy. First, we determine the individual prediction of a feature by comparing its weight (w) against the prediction threshold (τ_{pref}). Specifically, a weight $w \geq \tau_{pref}$ indicates an off-chip prediction, whereas $w < \tau_{pref}$ indicates an on-chip prediction. The confidence learning mechanism employs a per-entry Finite State Machine (FSM) within the confidence tables to dynamically update confidence values. The FSM rewards consistency by tracking the number of consecutive correct or incorrect predictions. Specifically, streaks of 1, 2, and 3 consecutive correct predictions transition the FSM to states SP_1, SP_2, and SP_3, which increment the confidence value by 1, 2, and 3, respectively. Conversely, streaks of 1, 2, and 3 consecutive incorrect predictions transition the FSM to states SN_1, SN_2, and SN_3, which

decrement the confidence value by 1, 2, and 3, respectively. Figure 7 shows the state transitions of the FSM. This FSM-based confidence learning mechanism aligns each feature's confidence with its prediction consistency, thereby gradually reinforcing reliable features while suppressing unstable ones across different program phases.

5.1.3 Feature Selection. The selection of features is paramount to the prediction performance of OPP. A carefully curated feature set enables OPP to capture complex memory access patterns, thereby significantly improving both prediction precision and coverage. To this end, we introduce an automated, performance-driven methodology to identify a set of features for OPP.

Leveraging our domain expertise, we initially select a candidate set of 17 features that can correlate well with whether prefetch requests go off-chip. Table 1 shows this initial feature set.

This performance-driven methodology is guided by the F1 score (shown in Eq. 1) and employs iterative evaluation to find a feature set for OPP. In the first iteration, we configure OPP using each of the 17 initial features individually and evaluate their F1 scores across 15 randomly selected workload traces. We retain the top-10 features yielding the highest F1 scores for the second iteration. In the second iteration, we generate feature combinations by pairing each of the top-10 features from the previous iteration with every feature from the initial feature set, subsequently evaluating their F1 scores. We again select the top-10 feature combinations providing the highest F1 scores for the third iteration. This iterative process repeats until the improvement in F1 score between successive iterations falls below 3%. Table 2 lists the final set of features selected by this performance-driven methodology.

$$\text{F1 score} = \frac{2 \times \text{Precision} \times \text{Coverage}}{\text{Precision} + \text{Coverage}} \quad (1)$$

Rationale for selected features. Each selected feature captures the likelihood of a prefetch request going off-chip by reflecting a different aspect of program context. The specific rationale for each selected feature is detailed below.

(1) Triggering PC ⊕ cacheline offset. This feature is computed by performing a bitwise XOR on the triggering PC (i.e., the PC of the instruction that triggered the current prefetch request) and the cacheline offset of the prefetch address within the physical page. The goal of this feature is to learn the likelihood of a prefetch request going off-chip when given a triggering PC and the cacheline offset accessed by the prefetch request. Utilizing the cacheline offset information enables this feature to apply learned knowledge to different physical pages.

(2) Triggering PC + first touch. We compute this feature by embedding the value of the first touch feature into the most-significant bit (MSB) position of the triggering PC. The first touch feature is a binary value indicating whether a cacheline has been recently touched by either the demand load or prefetch requests. The first touch feature value is obtained from a small prefetch-aware page buffer that tracks accessed cachelines from the last 64 physical pages. Each prefetch-aware page buffer entry maintains two fields: a physical page tag and a 64-bit bitmap, where each bit tracks the access status of a cacheline within the physical page. For every demand load or prefetch request, OPP queries the prefetch-aware page buffer using the request's physical page number. If a matching entry is found, the bit corresponding to the cacheline offset in the matching entry's bitmap is retrieved to serve as the first touch feature value. A set (or unset) bit indicates that the cacheline has (not) been recently accessed by the demand load or prefetch request. Crucially, if the bit is unset, OPP immediately sets the bit. The first touch feature provides an approximation of cacheline reuse within a short temporal window.

(3) Physical page number + first touch and (4) cacheline offset + first touch. These two features are similar to the triggering PC + first touch feature, except that they respectively learn the likelihood of a prefetch request going off-chip when a specific physical page number or cacheline offset has been recently accessed by either demand loads or prefetch requests.

(5) Last-4 prefetch physical addresses. This feature is computed by performing a shifted-XOR operation on the last four prefetch physical addresses. It represents the recent memory access pattern of the prefetcher and correlates the recurrence of that pattern with the likelihood of observing the prefetch request going off-chip.

(6) Last-4 load physical addresses. This feature is computed in a similar manner to the Last-4 prefetch physical addresses feature, but uses the physical addresses of the last four demand loads instead. It represents the program's recent memory access pattern and establishes a correlation between the recurrence of this pattern and the likelihood of a prefetch request going off-chip.

5.1.4 Threshold Tuning. OPP has four tunable threshold parameters: the activation threshold (τ_{act}), the prediction threshold (τ_{pref}) for individual feature prediction, the positive saturation threshold (T_{PO}), and the negative saturation threshold (T_{NE}). Proper tuning of these four threshold parameters is critical to OPP's prediction performance, as both precision and coverage of OPP are sensitive to their values.

We employ a three-step grid search methodology [6] to tune each of the four threshold parameters. First, we uniformly sample the search space of a parameter using a fixed step size. Specifically, the grid step size is set to 5 for thresholds τ_{act}, T_{PO}, and T_{NE}, and 1 for τ_{pref}. Second, we evaluate these candidates on 15 randomly selected workload traces, retaining the top-10 values that yield the highest performance. Finally, we evaluate these top candidates across all single-core workload traces and select the parameter value that maximizes the geometric mean performance improvement. Table 2 lists the final tuned threshold values.

5.2 Artemis Datapath Design

In this section, we detail the key changes introduced to the on-chip cache access datapath for incorporating Artemis. First, we describe how the Artemis architecture issues an Artemis request directly to the main memory controller. Second, we show how a regular prefetch request that misses in the on-chip cache hierarchy waits for an in-flight Artemis request. Finally, we discuss how data fetched by Artemis requests is correctly returned to the on-chip cache hierarchy while maintaining cache coherence.

5.2.1 Issuing an Artemis Request. For each prefetch request predicted to go off-chip, Artemis directly issues an Artemis request to the main memory controller (step ❷ in Figure 5), which then

Table 1: The initial set of features used for OPP prediction. ⊕ denotes a bitwise XOR operation.

Features with data-flow information	Features with control-flow information
1. Prefetch physical address	10. Triggering PC
2. Physical page number	11. Triggering PC ⊕ prefetch physical address
3. Page offset	12. Triggering PC ⊕ physical page number
4. Cacheline offset in page	13. Triggering PC ⊕ page offset
5. First touch	14. Triggering PC ⊕ cacheline offset
6. Physical page number + first touch	15. Triggering PC + first touch
7. Cacheline offset + first touch	16. Last-4 load PCs
8. Last-4 prefetch physical addresses	17. Last-4 PCs
9. Last-4 load physical addresses	

Table 2: Configuration Parameters of OPP.

Selected features	• Triggering PC ⊕ cacheline offset • Triggering PC + first touch • Physical page number + first touch • Cacheline offset + first touch • Last-4 prefetch physical addresses • Last-4 load physical addresses
Threshold values	$\tau_{act} = -125$, $\tau_{pref} = -3$, $T_{PO} = 65$, $T_{NE} = -60$

Table 3: Storage Overhead of Artemis.

Structure	Description	Size
OPP	• Weight tables: 3.21 KB Cacheline offset + first touch: 128 x 5b Each of the other features: 1024 x 5b • Confidence tables: 2.57 KB Cacheline offset + first touch: 128 x 4b Each of the other features: 1024 x 4b • Finite state machine: 1.93 KB • Prefetch-aware page buffer: 0.63 KB	8.34 KB
PQ metadata	Hashed feature: 57b; Prediction: 1b; First touch: 1b; Feature weight sum: 7b;	1.04 KB
Total		9.38 KB

enqueues the request into its read queue (RQ). Artemis requests are serviced by the main memory controller according to its scheduling policy, while regular prefetch requests concurrently access the on-chip cache hierarchy.

5.2.2 Waiting for Artemis requests. If Artemis's off-chip prediction is correct, the regular prefetch request eventually misses in the on-chip cache hierarchy and queries the main memory controller's RQ for any in-flight access targeting the same physical address. If a match is found, the regular prefetch request waits for the in-flight Artemis request to finish. Otherwise, the regular prefetch request proceeds to access main memory.

5.2.3 Returning Data to the On-Chip Cache Hierarchy. Upon the return of an Artemis request from main memory, Artemis inspects the RQ of the main memory controller. If there is a regular prefetch request targeting the same address waiting for the Artemis request, the fetched data is returned to the on-chip cache hierarchy. Conversely, if no matching prefetch request is waiting, Artemis discards the data and precludes its insertion into the cache hierarchy, thereby strictly preserving on-chip cache hierarchy coherence.

5.3 Implementation Cost of Artemis.

5.3.1 Artemis Storage Overhead. Table 3 shows the storage overhead of Artemis. Artemis requires only 9.38 KB of storage per processor core. OPP consumes 8.34 KB, while the metadata stored in the PQ for OPP training consumes 1.04 KB.

5.3.2 Additional Hardware Overhead. OPP utilizes narrow-bit integer arithmetic (six 5-bit × 4-bit multipliers and one adder), incurring negligible dynamic power. Among them, the multipliers are utilized to perform multiplication between weight values and confidence values, while the adder is employed to sum the results of these multiplications. The prediction mechanism of Artemis operates in parallel with regular LLC accesses, thereby avoiding critical path interference. Consequently, the integration of Artemis imposes no impact on the processor's existing timing constraints. Furthermore, in the event of an incorrect prediction, Artemis discards unnecessary prefetch data to maintain strict on-chip cache coherence. This design entirely bypasses modifications to existing coherence protocols, drastically reducing both integration effort and verification complexity.

6 Experimental Methodology

6.1 Simulation Methodology

We evaluate Artemis using ChampSim [12, 19], a trace-driven simulator that models an out-of-order processor core. We model a simulated system with reference to the Intel Panther Lake [11, 43, 48]. Table 4 presents the configuration parameters of the simulated system. Unless otherwise specified, the baseline system referred to in this paper refers to the system presented in Table 4. We use 100M [23] instructions for warm-up and another 100M [23] instructions for simulation in both single-core and multi-core evaluations. For multi-core evaluation, if a processor core finishes running early, it will rerun the workload traces until each processor core has executed at least 100M instructions.

Table 4: System Configuration.

Core	5 GHz, 8-wide out-of-order, 6-stage pipeline, 576-entry reorder buffer (ROB), hashed-perceptron branch predictor,
L1 ITLB	4-way, 64-entry, LRU, 8-entry MSHR, 1-cycle
L1 DTLB	4-way, 64-entry, LRU, 8-entry MSHR, 1-cycle
L2 TLB	12-way, 1536-entry, LRU, 16-entry MSHR, 8-cycle
L1I Cache	8-way, 64 KB, LRU, 10-entry MSHR, 5-cycle
L1D Cache	12-way, 192 KB, LRU, 10-entry MSHR, 8-cycle
L2 Cache	16-way, 3 MB, LRU, 16-entry MSHR, 40-cycle
LLC	Shared, 12-way, 3.5 MB per core, LRU, 64-entry MSHR, 55-cycle, Bingo [3]/Pythia [7]/SPP-PPF [9]/ RICH [1] prefetcher, Up to 18 MB total
DRAM	DDR5 SDRAM, 6400 MTPS, single-core data-rate: 51.2 GB/s per core n-core data-rate: (51.2/n) GB/s per core tCAS = tRCD = tRP = 12.5 ns
Artemis	Artemis request issue latency: 5-cycle

6.2 Workloads

Our evaluation considers a diverse set of memory-intensive workloads drawn from SPEC CPU 2006 [13], SPEC CPU 2017 [14], Ligra [40], CVP [33], and the GAP [4] benchmark suite, as summarized in Table 5. In our evaluation, we only consider workload traces that have at least 3 LLC MPKI in the simulated system. This is because, for workload traces with an LLC MPKI of at least 3, Artemis can hide the LLC access latency for more off-chip prefetch requests, thereby providing more significant performance gains to the processor. In total, we evaluate Artemis using 80 single-core workload traces from 52 workloads. For each iteration of feature selection and threshold tuning, we randomly choose 15 workload traces from Table 5 for evaluation.

In our multi-core simulations, we construct both homogeneous and heterogeneous workload trace mixes. For the four-core homogeneous simulation, each processor core runs the same workload trace. For the heterogeneous simulation, we randomly select four workload traces from the single-core workload trace list and each is assigned to a different core. In total, we evaluate Artemis using 80 homogeneous and 80 heterogeneous four-core workloads.

6.3 Evaluated System Configurations

To comprehensively evaluate Artemis, we compare it against several state-of-the-art off-chip predictors, and evaluate its performance when integrated with various state-of-the-art LLC prefetchers. Table 6 presents the storage overhead for all evaluated configurations.

6.3.1 Various Off-chip Predictors. For off-chip predictors, we compare OPP with three predictors: HMP [53], CATP, and FLP [23].

HMP is a hybrid predictor composed of three sub-predictors. For the prefetch request generated by the LLC prefetcher, each sub-predictor uses its own prediction mechanism to make predictions. HMP considers the prediction results of each sub-predictor and selects the majority prediction.

We design CATP inspired by prior cacheline address tracking-based prediction mechanisms [6, 22, 28, 36]. CATP tracks the partial tag of the cacheline address that is likely to reside in the LLC using a dedicated metadata structure. For every LLC fill or eviction, the partial tag of the cacheline address is inserted into or evicted from CATP's metadata structure. To predict whether a prefetch request will miss in the LLC, CATP queries the metadata structure with the partial tag of the prefetch request. If the tag is not found in the metadata structure, CATP predicts the prefetch request will miss in the LLC.

FLP is a state-of-the-art perceptron-based off-chip load predictor. In our experiments, we use it to predict whether prefetch requests will miss in the LLC.

6.3.2 Various LLC Prefetchers. We evaluate Artemis combined with four state-of-the-art LLC prefetchers (presented in Table 6) to verify its synergistic gains and compatibility across different prefetching mechanisms.

7 Evaluation

7.1 Prediction Performance Analysis

7.1.1 Prediction Performance of OPP. We evaluate the prediction precision and coverage for OPP and various predictors, and compute their prediction performance using the F1 score, as defined in Equation (1). Figure 8 shows the precision, coverage, and F1 scores achieved by OPP and the evaluated predictors when combined with different LLC prefetchers.

We make three key observations from Figure 8. First, compared to other predictors, our proposed predictors achieve significantly higher precision and F1 score. Specifically, when integrated with the Bingo, Pythia, SPP-PPF, and RICH prefetchers, OPP achieves an average precision (F1 score) of 84.3% (0.854), 83.6% (0.840), 85.0% (0.857), and 86.2% (0.871), respectively. Across all evaluated prefetchers, OPP achieves an average precision of 84.8% and an average F1 score of 0.855, substantially outperforming HMP (30.7% and 0.228), CATP (14.2% and 0.247), and FLP (60.3% and 0.588). Second, across all evaluated prefetchers, OPP achieves an average coverage of 86.2%, substantially outperforming HMP (18.1%) and FLP (57.4%), and is second only to CATP (95.7%). Although CATP provides the highest prediction coverage, it suffers from the lowest precision. Poor precision triggers a large number of unnecessary Artemis requests, thereby wasting main memory bandwidth. Third, OPP achieves substantially higher precision and F1 scores across all benchmark suites and evaluated LLC prefetchers compared to HMP, CATP, and FLP. This validates both the effectiveness and the compatibility of our proposed OPP predictor.

The performance advantage of OPP over FLP, both being perceptron-based predictors, stems from the following factors. First, OPP employs the triggering PC feature to correlate specific prefetch requests. Second, it employs the first touch feature to capture the memory access behavior of prefetch requests. Third, OPP utilizes a confidence learning mechanism to dynamically reinforce (attenuate) the contribution of useful (useless) features. Figure 9 compares the prediction performance of triggering PC-related features against PC-related features, as well as first touch-related features against first access-related features. We make two key observations from Figure 9. First, triggering PC-related features significantly outperform PC-related features in terms of average precision, average

Table 5: Workloads Used in the Evaluation and Their Input Configurations.

Suite	Workloads	Traces	Parallelism Type	Description	Input Workloads	LLC MPKI
SPEC06	14	20	Multi-programmed	Independent tasks	cactusADM, mcf, lbm, ...	≥ 3
SPEC17	10	13	Multi-programmed	Independent tasks	fotonik3d, mcf, pop2, ...	≥ 3
Ligra	6	10	Internal Multi-threading	Thread-level parallelism	Triangle, PageRank, BFS, ...	≥ 3
CVP	17	17	Instruction Stream	Diverse access patterns	server, integer, fp, ...	≥ 3
GAP	5	20	Internal Multi-threading	Thread-level parallelism	bc, bfs, cc, pr, sssp	≥ 3

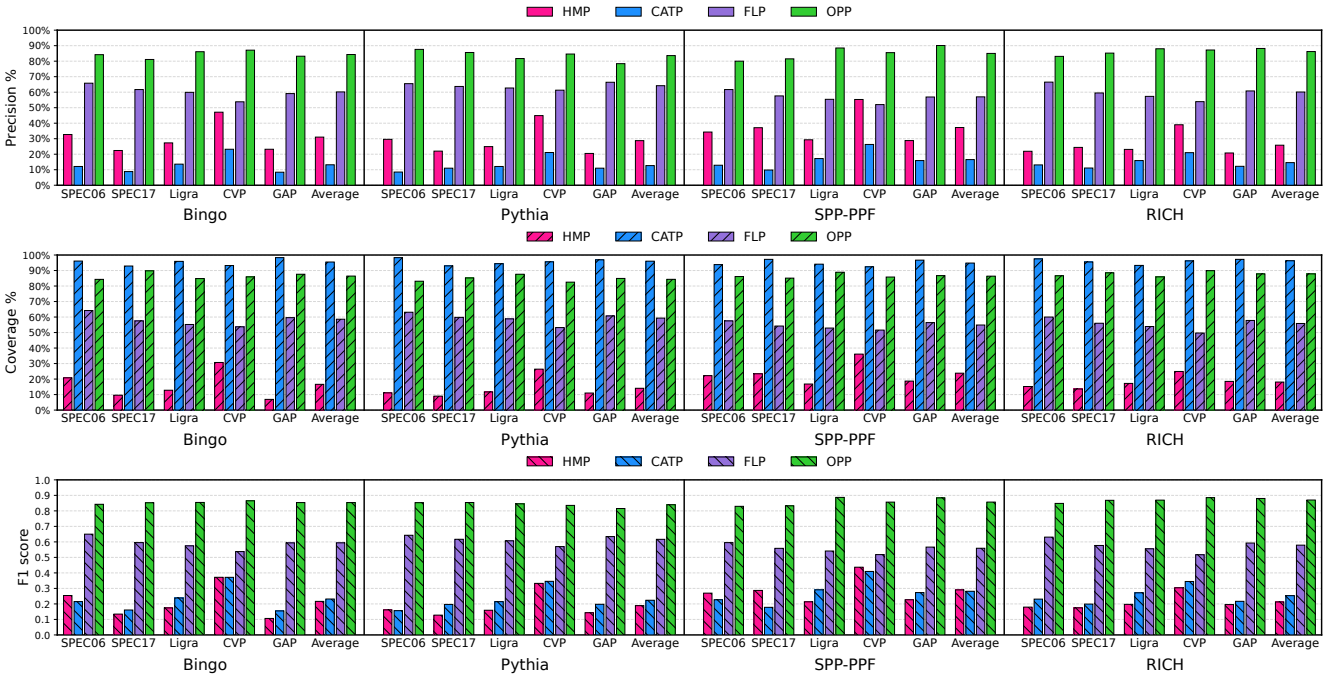

Figure 8: Evaluation of precision, coverage, and F1 score for various off-chip prefetch predictors.

Table 6: Storage Overhead of Evaluated Configurations.

Off-chip Predictors	Size	LLC Prefetchers	Size
OPP (this work)	9.38 KB	Bingo	46 KB
HMP	11 KB	Pythia	25.5 KB
CATP	1536 KB	SPP-PPF	39.3 KB
FLP	3.21 KB	RICH	175.3 KB

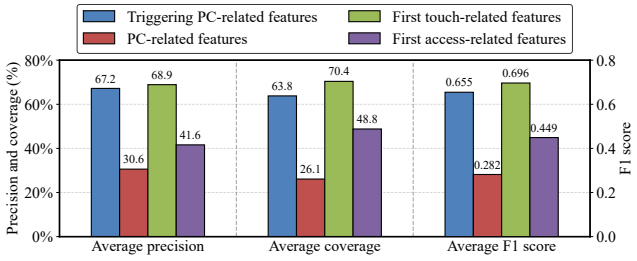

Figure 9: Prediction performance comparison across triggering PC-related features, PC-related features, first touch-related features, and first access-related features.

coverage, and average F1 score. Specifically, the former achieves an average precision, coverage, and F1 score of 67.2%, 63.8%, and 0.655, respectively, whereas the latter only yields 30.6%, 26.1%, and 0.282. Second, first touch-related features also exhibit significantly superior performance compared to first access-related features. The former reaches an average precision, coverage, and F1 score of 68.9%, 70.4%, and 0.696, surpassing the latter by 27.3%, 21.6%, and 0.247, respectively.

Based on our experimental results, we conclude that OPP provides the best prediction performance for off-chip prefetch prediction compared to other predictors.

7.2 Single-core Performance Evaluation

7.2.1 Single-core Performance Improvement. Figure 10(a) shows the single-core performance improvements provided by standalone prefetchers versus those combined with Artemis, normalized to the baseline (Panther Lake) systems with no prefetcher. Figure 10(b)

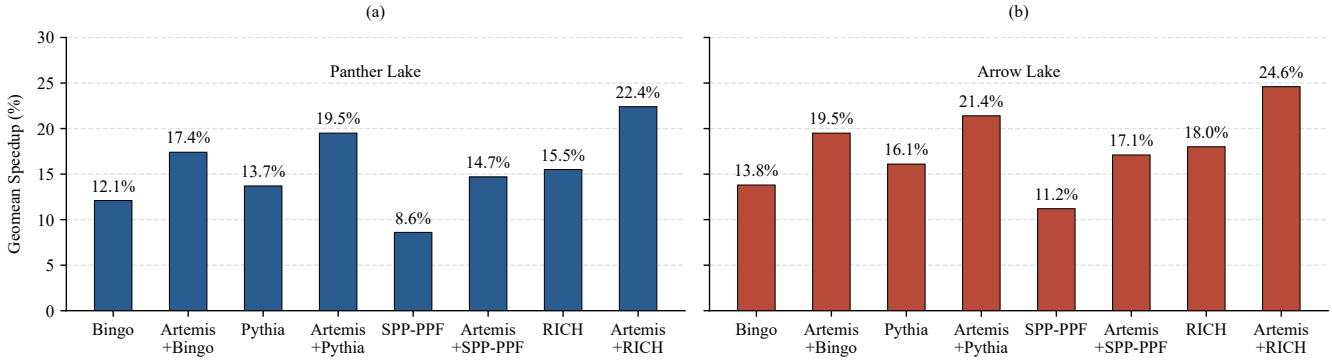

Figure 10: Single-core speedup across different microarchitectures: (a) Panther Lake, (b) Arrow Lake.

shows the same scenarios on a simulated system with reference to Arrow Lake [10, 49].

We make two key observations from Figure 10. First, combining Artemis with the Bingo, Pythia, SPP-PPF, and RICH prefetchers provides single-core geomean speedups of 17.4%, 19.5%, 14.7%, and 22.4% under Panther Lake [11, 43, 48], and 19.5%, 21.4%, 17.1%, and 24.6% under Arrow Lake [10, 49], respectively. Second, regardless of the prefetcher it is combined with, Artemis consistently provides higher single-core geomean speedups than the standalone prefetchers under Panther Lake baselines. On average (geomean), the integration of Artemis provides single-core speedup improvements of 5.3%, 5.8%, 6.1%, and 6.9% over the standalone Bingo, Pythia, SPP-PPF, and RICH prefetchers under Panther Lake baseline system, respectively. Artemis provides a similarly significant single-core performance advantage under the Arrow Lake baseline configuration.

Too-early prefetching causes prefetched data to be prematurely evicted before being actually accessed, which not only induces cache pollution but also wastes valuable main memory bandwidth. Our evaluation under single-core configurations demonstrates that Artemis introduces a negligible number of additional too-early prefetches, ranging from merely 0.01 to 0.03 per kilo instructions across all four LLC prefetchers.

Based on our experimental results, we conclude that combining Artemis with various state-of-the-art LLC prefetchers consistently provides higher single-core performance improvements than standalone LLC prefetchers across both Panther Lake and Arrow Lake configurations, demonstrating both the effectiveness of Artemis and its compatibility with modern processors and LLC prefetchers.

7.2.2 Impact of Prediction Mechanisms on Single-core Performance. In this section, we evaluate the impact of different off-chip prefetch predictors on the single-core performance improvements provided by Artemis. Figure 11 shows the single-core geomean performance improvements achieved by combining LLC prefetchers with Artemis equipped with different off-chip prefetch predictors.

We make two key observations from Figure 11. First, Artemis+OPP provides the highest single-core geomean performance improvement across all LLC prefetchers compared to Artemis+HMP, Artemis+CATP, and Artemis+FLP. Second, regardless of the off-chip prefetch predictor it is integrated with, Artemis consistently

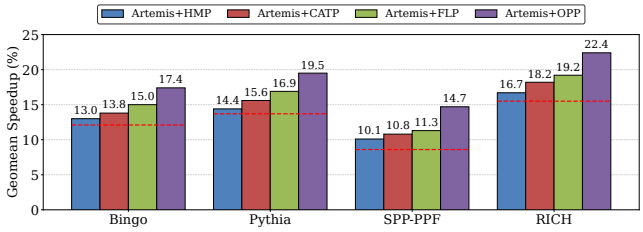

Figure 11: Single core performance improvement provided by Artemis with various off-chip prefetch predictors across various LLC prefetchers. The red dashed line represents the single-core performance improvement provided by the standalone LLC prefetcher.

provides higher single-core geomean performance improvements than the standalone LLC prefetchers.

We draw two conclusions. First, since OPP provides the best prediction performance compared to other predictors, Artemis equipped with OPP provides the highest single-core geomean performance improvement. Second, regardless of the predictor used, integrating Artemis with LLC prefetchers consistently provides higher single-core geomean performance improvements than standalone LLC prefetchers, demonstrating both the effectiveness of Artemis and its compatibility with various off-chip prefetch predictors.

7.3 Multi-core Performance Evaluation

Figure 12 shows the four-core performance improvements provided by the prefetchers and Artemis combined with these prefetchers, normalized to the baseline system with no prefetcher. Figure 13 shows the geomean performance improvement provided by the LLC prefetchers and Artemis combined with these prefetchers across 2-, 4-, 8-, 16-, 32-, and 64-core configurations, normalized to the baseline system with no prefetcher.

We make two key observations from Figure 12 and Figure 13. First, in the 4-core configuration, Artemis combined with LLC prefetchers provides significantly higher geomean speedups compared to standalone LLC prefetchers. Specifically, Artemis-augmented versions of Bingo, Pythia, SPP-PPF, and RICH deliver geomean

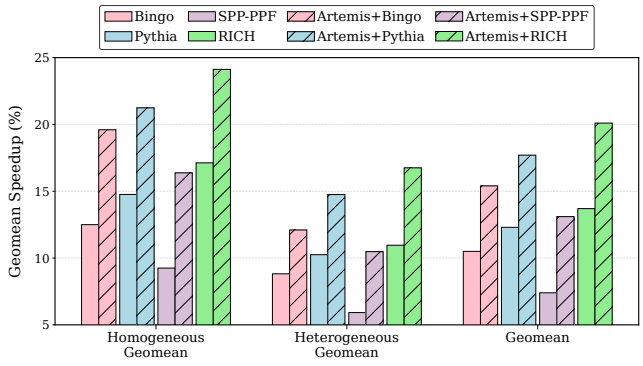

Figure 12: Geomean performance improvement in the four-core scenario.

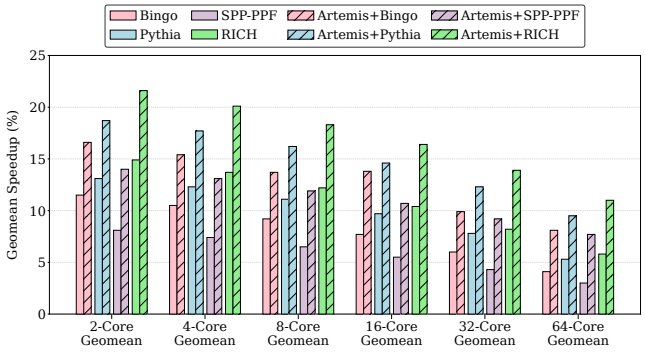

Figure 13: Geomean performance improvement in 1-, 2-, 4-, 8-, 16-, 32-, and 64-core configurations.

speedups of 15.4%, 17.7%, 13.1%, and 20.1% in the four-core configuration, outperforming their standalone counterparts by 4.9%, 5.4%, 5.7%, and 6.4%, respectively. Second, Artemis consistently achieves higher geomean speedups than standalone LLC prefetchers across 2-, 4-, 8-, 16-, 32-, and 64-core configurations. We conclude that the Artemis architecture provides consistent performance improvements across different core counts.

7.4 Performance Sensitivity Analysis

7.4.1 Effect of Main Memory Bandwidth. Figure 14 shows the speedups of standalone LLC prefetchers and Artemis combined with LLC prefetchers over the baseline system with no prefetcher in a single-core system, as the main memory bandwidth scales. We make two key observations from Figure 14. First, Artemis combined with LLC prefetchers consistently outperforms standalone LLC prefetchers in every main memory bandwidth configuration. Second, as the main memory bandwidth increases, the single-core speedups provided by Artemis integrated with LLC prefetchers also increase.

We conclude that Artemis combined with LLC prefetchers consistently maintains a single-core performance advantage over standalone LLC prefetchers, across both low and high memory bandwidth configurations.

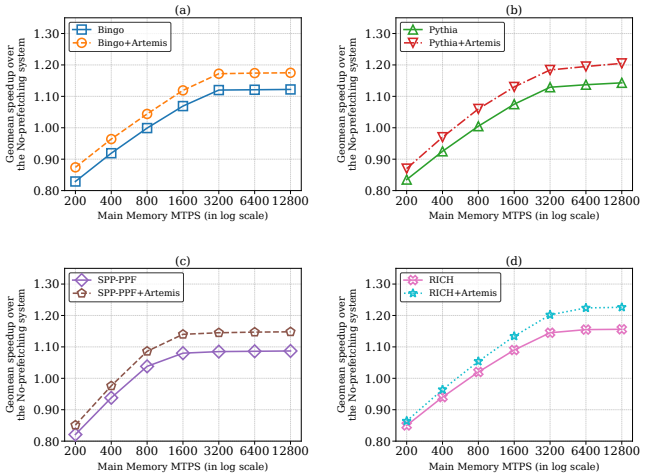

Figure 14: Single-core performance sensitivity to main memory bandwidth.

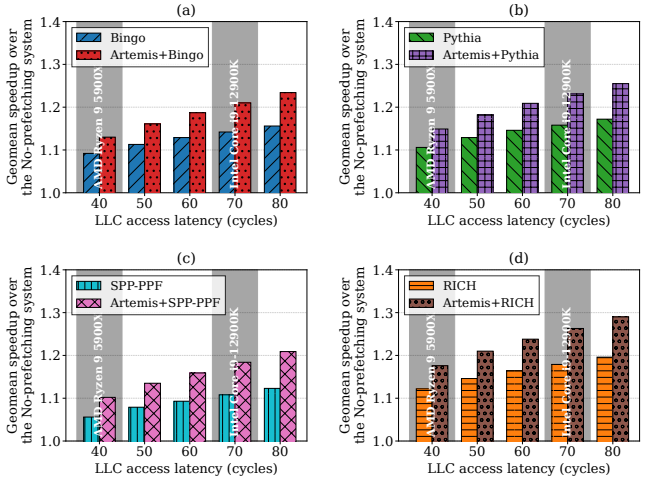

Figure 15: Single-core performance sensitivity to LLC access latency.

7.4.2 Effect of LLC Access Latency. Figure 15 shows the speedups of standalone LLC prefetchers and Artemis combined with LLC prefetchers over the baseline system with no prefetcher in a single-core system, as the LLC access latency increases. We make three key observations from Figure 15. First, Artemis combined with LLC prefetchers consistently outperforms standalone LLC prefetchers for every LLC access latency. Second, the single-core performance improvements provided by Artemis combined with LLC prefetchers increase as the LLC access latency increases. Third, the single-core performance advantage of Artemis combined with LLC prefetchers over standalone LLC prefetchers widens as the LLC access latency increases. Therefore, we posit that Artemis can provide higher performance improvements in future processors with longer LLC access latencies.

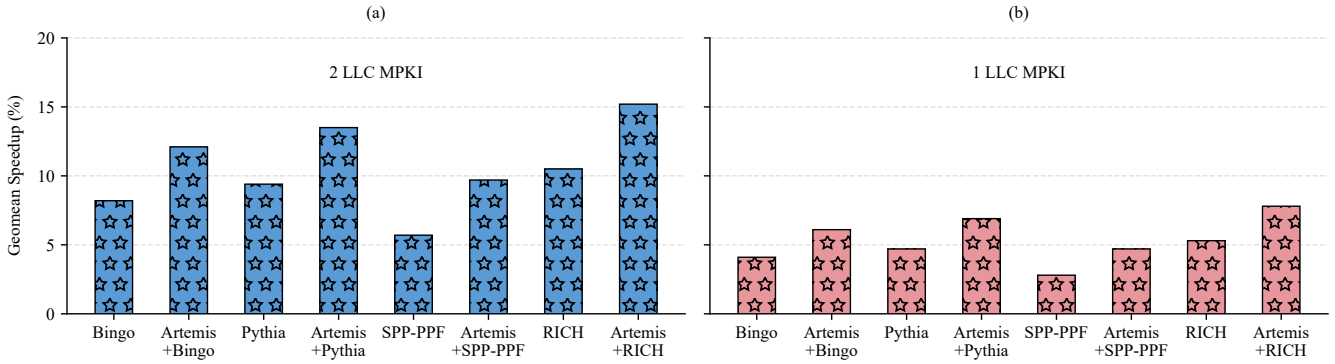

Figure 16: Single-core speedup of standalone and Artemis-combined LLC prefetchers. (a) Evaluated under 20 workload traces with 2 LLC MPKI; (b) Evaluated under 20 workload traces with 1 LLC MPKI.

7.4.3 Effect of 1 and 2 LLC MPKI Workloads on Performance. Figure 16 presents the single-core speedups of standalone and Artemis-combined LLC prefetchers over the baseline system with no prefetcher, evaluated on 20 randomly selected workload traces each with LLC MPKI values of 1 and 2. We make two observations from Figure 16. First, across both 1 and 2 LLC MPKI workload traces, the combination of Artemis with LLC prefetchers consistently outperforms standalone LLC prefetchers in single-core performance. Specifically, under 20 workload traces with 2 LLC MPKI, the integration of Artemis with Bingo, Pythia, SPP-PPF, and RICH prefetchers provides single-core performance improvements of 12.1%, 13.5%, 9.7%, and 15.2%, respectively, outperforming their standalone counterparts by 3.9%, 4.1%, 4.0%, and 4.7%. In addition, across 20 workload traces with 1 LLC MPKI, the combination of Artemis with LLC prefetchers provides single-core performance improvements ranging from 1.9% to 2.5% over their standalone counterparts. Second, across all evaluated LLC prefetchers, Artemis consistently provides lower single-core performance improvements on 1 LLC MPKI workload traces compared to those with 2 LLC MPKI. This is because lower LLC MPKI workloads limit the number of accurate prefetch requests that LLC prefetchers can issue, thereby reducing the memory access latency saved by both the LLC prefetchers and Artemis for the processor.

We conclude that, across both 2 LLC MPKI and 1 LLC MPKI workload traces, the combination of Artemis with LLC prefetchers consistently provides higher single-core performance improvements compared to standalone LLC prefetchers.

7.5 Main memory Access Overhead

Figure 17 shows the percentage increase in main memory requests for Artemis combined with LLC prefetchers and standalone LLC prefetchers over the baseline system with no prefetchers in the single-core configuration. We make the key observations from Figure 17. Regardless of the LLC prefetcher, Artemis combined with LLC prefetchers consistently exhibits a higher percentage increase in main memory requests compared to standalone prefetchers. Specifically, when combined with Bingo, Pythia, SPP-PPF, and RICH, Artemis incurs main memory request overheads that are 4.2%, 5.9%, 3.8%, and 7.5% higher than standalone LLC prefetchers,

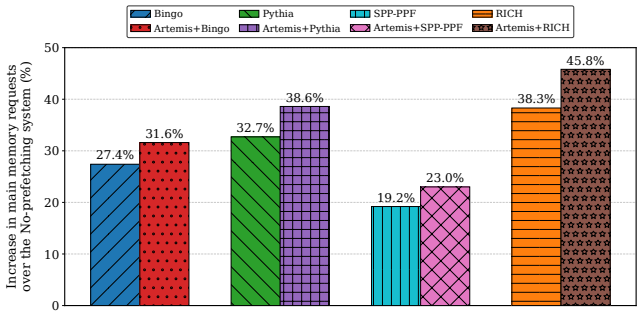

Figure 17: Overhead in the main memory requests.

respectively. This means that every 1% performance improvement provided by Artemis on top of Bingo, Pythia, SPP-PPF, and RICH comes at a cost of only 0.79%, 1.02%, 0.62%, and 1.09% additional overhead in main memory requests, respectively.

Furthermore, we evaluated the average percentage increase in main memory requests for Artemis across four prefetchers in 2-core, 4-core, 8-core, 16-core, 32-core, and 64-core configurations. Specifically, under 2-, 4-, 8-, 16-, 32-, and 64-core configurations, Artemis introduces an average increase in main memory requests of 5.6%, 6.0%, 6.5%, 7.2%, 8.3%, and 9.7% across the four baseline prefetchers, respectively.

We conclude that Artemis, benefiting from the high-accuracy prediction mechanism of OPP, provides significant performance improvements while incurring only a modest increase in main memory request overhead when combined with LLC prefetchers.

8 Related Work

Artemis is the first work to utilize a neural predictor to predict whether a prefetch request generated by the LLC prefetcher will miss in the LLC. When the prediction is correct, Artemis issues a speculative request to main memory with the same physical address as the off-chip prefetch request in advance, effectively removing LLC access latency from the critical path of off-chip prefetch requests. In this section, we discuss other related works that employ neural predictors.

Branch predictors. Branch predictors are responsible for predicting both the direction and the target address of a conditional branch in an instruction stream to enable the processor to speculatively execute instructions before the branch condition is resolved. Garza et al. [18] propose a bit-level perceptron-based indirect branch predictor (BLBP) that utilizes perceptron-based neural learning to predict individual bits of target addresses and selects the closest matching target from an indirect branch target buffer (IBTB) via cosine similarity. Zangeneh et al. [54] propose a CNN-based branch predictor named BranchNet that utilizes a compile-time trained convolutional neural network to identify complex, input-independent branch correlations and performs runtime inference to accurately predict hard-to-predict branches that are typically mispredicted by traditional predictors like TAGE [37]. Xiong et al. [50] propose a shallow online neural network (SONet) that utilizes a lightweight training and inference architecture to dynamically identify and offload hard-to-predict (H2P) branches from traditional predictors like TAGE-SC-L, effectively mitigating table entry allocation pressure and improving prediction accuracy through runtime neural learning.

Prefetch filters. Prefetch filters are responsible for predicting which prefetch requests generated by prefetchers are useless. By filtering out these requests, prefetch filters can alleviate cache pollution and conserve memory bandwidth. Bhatia et al. [9] propose a prefetch filter (PPF) that utilizes perceptron-based neural learning to predict and discard inaccurate prefetch requests generated by aggressive prefetchers, thereby mitigating cache pollution and conserving memory bandwidth without compromising overall prefetch coverage. Jamet et al. [23] proposed an adaptive prefetch filter (SLP) using the perceptron neural learning mechanism to effectively improve processor performance and mitigate energy overheads in memory-intensive applications. The primary distinction between SLP and PPF is that the SLP focuses on adaptive filtering of prefetch requests using physical address information, while the PPF adjusts filtering sensitivity based on the prefetcher's performance history and confidence.

Data prefetchers. Shi et al. [39] propose a Hierarchical Neural Model of Data Prefetching (Voyager) that utilizes a novel attention-based embedding layer to solve key challenges of data prefetching (class explosion and labeling) by separating addresses into pages and offsets, enabling the model to learn both delta and address correlations. Duong et al. [16] propose a new temporal prefetcher called Twilight, which improves upon the previous state-of-the-art neural prefetcher, Voyager, by introducing two abstraction layers: frequency-based candidate selection and behavioral clustering, thereby significantly reducing model size and inference latency while enhancing generalization capabilities. Tang et al. [41] propose a practical and intelligent first-level cache (L1D) data prefetcher called Amphi that integrates binarized temporal convolutional neural networks (TCN) to balance storage overhead and performance.

Reuse predictors. Reuse predictors are responsible for predicting whether a cache block will be accessed again before it is evicted, allowing for optimizations such as improved placement, replacement, and bypass. Teran et al. [44] propose a perceptron learning mechanism for reuse prediction that utilizes perceptron-based neural learning to combine multiple input features for cache block reuse prediction, effectively overcoming the limitations of previous methods in the LLC. Jiménez et al. [26] propose a Multiperspective Reuse Prediction (MRP) technique that utilizes perceptron-based neural learning to combine multiple parameterized features for cache block reuse prediction. Compared to the previous perceptron-based reuse predictor, MRP improves accuracy and efficiency by incorporating a wider variety of parameterized features and leveraging the cache block's placement position to implicitly record reuse predictions.

Off-chip load predictors. Off-chip load predictors are responsible for predicting whether a load request will miss in the entire on-chip cache hierarchy. Bera et al. [6] propose a perceptron-based off-chip load prediction mechanism called Hermes that accelerates long-latency load requests by predicting whether a load will miss the on-chip cache hierarchy, thereby allowing the processor to bypass cache lookups and initiate off-chip requests earlier. Jamet et al. [23] propose an off-chip load predictor called FLP using a perceptron-based neural learning mechanism. FLP differs from Hermes by integrating a selective delay mechanism to improve prediction accuracy.

Cache miss load predictors. Cache miss load predictors are responsible for predicting whether a load request will miss in a specific level of cache. Huang et al. [21] propose L1D and L2 miss load predictors—namely L1MP and L2MP—which leverage perceptron-based neural mechanisms. Unlike off-chip load request predictors, L1MP and L2MP introduce two novel features that capture the memory access behavior of prefetch requests to enhance their prediction performance.

9 Conclusion

We propose Artemis, a technique that improves processor performance by hiding the LLC access latency of accurate prefetch requests issued by LLC prefetchers. To enable Artemis, we introduce OPP, an off-chip prefetch predictor based on perceptron and confidence learning mechanisms, designed to predict whether prefetch requests generated by LLC prefetchers will miss in the LLC. Our evaluation across various benchmark suites demonstrates that Artemis consistently provides higher performance gains than standalone LLC prefetchers across diverse system configurations, while incurring only 9.38 KB of storage overhead and a modest increase in main memory request overhead.

10 Acknowledgments

We thank the anonymous reviewers for their helpful feedback on previous versions of this work. We are also deeply grateful to our shepherd for the invaluable guidance and support during the revision process. This work is supported by the National Natural Science Foundation of China under Grant U23A20301.

References

[1] Ningzhi Ai, Wenjian He, Hu He, Jing Xia, Heng Liao, and Guowei Zhang. 2025. RICH Prefetcher: Storing Rich Information in Memory to Trade Capacity and Bandwidth for Latency Hiding. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 170–183.

[2] Sam Ainsworth and Lev Mukhanov. 2024. Triangel: A high-performance, accurate, timely on-chip temporal prefetcher. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1202–1216.

[3] Mohammad Bakhshalipour, Mehran Shakerinava, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Bingo spatial data prefetcher. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE,

399–411.
[4] Scott Beamer, Krste Asanović, and David Patterson. 2015. The GAP benchmark suite. *arXiv preprint arXiv:1508.03619* (2015).
[5] Bradford M Beckmann and David A Wood. 2004. Managing wire delay in large chip-multiprocessor caches. In *37th International Symposium on Microarchitecture (MICRO-37'04)*. IEEE, 319–330.
[6] Rahul Bera, Konstantinos Kanellopoulos, Shankar Balachandran, David Novo, Ataberk Olgun, Mohammad Sadrosadat, and Onur Mutlu. 2022. Hermes: Accelerating long-latency load requests via perceptron-based off-chip load prediction. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–18.
[7] Rahul Bera, Konstantinos Kanellopoulos, Anant Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 1121–1137.
[8] Rahul Bera, Anant V Nori, Onur Mutlu, and Sreenivas Subramoney. 2019. Dspatch: Dual spatial pattern prefetcher. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 531–544.
[9] Eshan Bhatia, Gino Chacon, Seth Pugsley, Elvira Teran, Paul V Gratz, and Daniel A Jiménez. 2019. Perceptron-based prefetch filtering. In *Proceedings of the 46th International Symposium on Computer Architecture*. 1–13.
[10] Chips and Cheese. 2024. Examining Intel's Arrow Lake, at the System Level. Retrieved May 26, 2026 from https://chipsandcheese.com/p/examining-intels-arrow-lake-at-the
[11] ComputerBase. 2025. Intel Panther Lake in detail: This is Core Ultra 300 with Intel 18A, new cores and Xe3. Retrieved May 26, 2026 from https://www.computerbase.de/artikel/prozessoren/intel-panther-lake-details.94482/#abschnitt_drei_chiptiles_von_intel_und_tsmc
[12] ChampSim Contributors. 2021. ChampSim: An Open-source Trace based Simulator. Retrieved October 6, 2025 from https://github.com/ChampSim/ChampSim
[13] The Standard Performance Evaluation Corporation. 2006. SPEC CPU 2006. Retrieved October 6, 2025 from https://www.spec.org/cpu2006/
[14] The Standard Performance Evaluation Corporation. 2017. SPEC CPU 2017. Retrieved October 6, 2025 from https://www.spec.org/cpu2017/
[15] Yujie Cui, Wei Chen, Xu Cheng, and Jiangfang Yi. 2024. Hyperion: A highly effective page and pc based delta prefetcher. *ACM Transactions on Architecture and Code Optimization* 21, 4 (2024), 1–27.
[16] Quang Duong, Akanksha Jain, and Calvin Lin. 2024. A new formulation of neural data prefetching. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1173–1187.
[17] Gelin Fu, Tian Xia, Zhongpei Luo, Ruiyang Chen, Wenzhe Zhao, and Pengju Ren. 2024. Differential-matching prefetcher for indirect memory access. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 439–453.
[18] Elba Garza, Samira Mirbagher-Ajorpaz, Tahsin Ahmad Khan, and Daniel A Jiménez. 2019. Bit-level perceptron prediction for indirect branches. In *Proceedings of the 46th International Symposium on Computer Architecture*. 27–38.
[19] Nathan Gober, Gino Chacon, Lei Wang, Paul V Gratz, Daniel A Jimenez, Elvira Teran, Seth Pugsley, and Jinchun Kim. 2022. The championship simulator: Architectural simulation for education and competition. *arXiv preprint arXiv:2210.14324* (2022).
[20] Nikos Hardavellas, Michael Ferdman, Babak Falsafi, and Anastasia Ailamaki. 2009. Reactive NUCA: near-optimal block placement and replication in distributed caches. In *Proceedings of the 36th annual international symposium on Computer architecture*. 184–195.
[21] Zhengwei Huang, Wei Guo, and Yongwen Wang. 2026. Apollo: Accelerating Load Requests via Multi-Level Cache Miss Load Prediction. *ACM Transactions on Architecture and Code Optimization* (2026).
[22] Majid Jalili and Mattan Erez. 2022. Reducing load latency with cache level prediction. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 648–661.
[23] Alexandre Valentin Jamet, Georgios Vavouliotis, Daniel A Jiménez, Lluc Alvarez, and Marc Casas. 2024. A two level neural approach combining off-chip prediction with adaptive prefetch filtering. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 528–542.
[24] He Jiang, Liuwei Fu, Dong Liu, Zhilei Ren, Yuting Chen, and Lei Qiao. 2025. TRACED: A Temporal Graph Neural Networks-based Model for Data Prefetching. *ACM Transactions on Architecture and Code Optimization* 22, 3 (2025), 1–25.
[25] Shizhi Jiang, Qiusong Yang, and Yiwei Ci. 2022. Merging similar patterns for hardware prefetching. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1012–1026.
[26] Daniel A Jiménez and Elvira Teran. 2017. Multiperspective reuse prediction. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. 436–448.
[27] Jinchun Kim, Seth H Pugsley, Paul V Gratz, AL Narasimha Reddy, Chris Wilkerson, and Zeshan Chishti. 2016. Path confidence based lookahead prefetching. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
[28] Gabriel H Loh and Mark D Hill. 2011. Efficiently enabling conventional block sizes for very large die-stacked DRAM caches. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*. 454–464.
[29] Onur Mutlu, Hyesoon Kim, and Yale N Patt. 2005. Techniques for efficient processing in runahead execution engines. In *32nd International Symposium on Computer Architecture (ISCA'05)*. IEEE, 370–381.
[30] Onur Mutlu, Jared Stark, Chris Wilkerson, and Yale N Patt. 2003. Runahead execution: An alternative to very large instruction windows for out-of-order processors. In *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings*. IEEE, 129–140.
[31] Biswabandan Panda. 2023. Clip: Load criticality based data prefetching for bandwidth-constrained many-core systems. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 714–727.
[32] Leeor Peled, Uri Weiser, and Yoav Etsion. 2019. A neural network prefetcher for arbitrary memory access patterns. *ACM Transactions on Architecture and Code Optimization (TACO)* 16, 4 (2019), 1–27.
[33] Arthur Perais. 2023. Second Championship Value Prediction (CVP-2). Retrieved March 21, 2026 from https://www.microarch.org/cvp1/cvp2/
[34] THE FPS REVIEW. 2021. Intel Core i5-12600K DDR4 Alder Lake CPU Review. Retrieved January 11, 2026 from https://www.thefpsreview.com/2021/12/08/intel-core-i5-12600k-ddr4-alder-lake-cpu-review/6/
[35] THE FPS REVIEW. 2025. The Intel 12th Gen Core i9-12900K Review. Retrieved January 26, 2026 from https://www.thefpsreview.com/2022/02/08/intel-core-i9-12900k-vs-amd-ryzen-9-5900x-performance-review/6/#fps-toc-h2-5
[36] Andreas Sembrant, Erik Hagersten, and David Black-Schaffer. 2014. The direct-to-data (d2d) cache: Navigating the cache hierarchy with a single lookup. *ACM SIGARCH Computer Architecture News* 42, 3 (2014), 133–144.
[37] André Seznec. 2016. Tage-sc-l branch predictors again. In *5th JILP Workshop on Computer Architecture Competitions (JWAC-5): Championship Branch Prediction (CBP-5)*.
[38] Mehran Shakerinava, Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Multi-lookahead offset prefetching. *The Third Data Prefetching Championship* 2019 (2019).
[39] Zhan Shi, Akanksha Jain, Kevin Swersky, Milad Hashemi, Parthasarathy Ranganathan, and Calvin Lin. 2021. A hierarchical neural model of data prefetching. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 861–873.
[40] Julian Shun and Guy E Blelloch. 2013. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*. 135–146.
[41] Xuan Tang, Zicong Wang, Shuiyi He, Dezun Dong, and Xiangke Liao. 2025. Amphi: Practical and Intelligent Data Prefetching for the First-Level Cache. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–2.
[42] David Tarjan and Kevin Skadron. 2005. Merging path and gshare indexing in perceptron branch prediction. *ACM transactions on architecture and code optimization (TACO)* 2, 3 (2005), 280–300.
[43] TechPowerUp. 2025. Intel Panther Lake Technical Deep Dive. Retrieved May 26, 2026 from https://www.techpowerup.com/review/intel-panther-lake-technical-deep-dive/5.html
[44] Elvira Teran, Zhe Wang, and Daniel A Jiménez. 2016. Perceptron learning for reuse prediction. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
[45] Georgios Vavouliotis, Gino Chacon, Lluc Alvarez, Paul V Gratz, Daniel A Jiménez, and Marc Casas. 2022. Page size aware cache prefetching. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 956–974.
[46] Zhengrong Wang, Jian Weng, Jason Lowe-Power, Jayesh Gaur, and Tony Nowatzki. 2021. Stream floating: Enabling proactive and decentralized cache optimizations. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 640–653.
[47] Wccftech. 2021. Surprisingly High Latency Discovered in Alder Lake. Retrieved May 26, 2026 from https://bit.ly/3RhlK8z
[48] Wccftech. 2025. Intel Panther Lake Deep-Dive: 18A Compute Tile With Cougar Cove P-Cores, Darkmont E-Cores. Retrieved May 26, 2026 from https://wccftech.com/intel-panther-lake-deep-dive-18a-compute-tile-cougar-cove-p-cores-darkmont-e-cores/
[49] Wikipedia. 2024. Arrow Lake (microprocessor). Retrieved May 26, 2026 from https://en.wikipedia.org/wiki/Arrow_Lake_(microprocessor)
[50] Zhenxuan Xiong, Libo Huang, Ling Yang, Hui Guo, Junhui Wang, Zhong Zheng, Songwen Pei, Gang Chen, and Yongwen Wang. 2025. SONet: Towards Practical Online Neural Network for Enhancing Hard-to-Predict Branches. In *European Conference on Parallel Processing*. Springer, 89–102.
[51] Feng Xue, Chenji Han, Xinyu Li, Junliang Wu, Tingting Zhang, Tianyi Liu, Yifan Hao, Zidong Du, Qi Guo, and Fuxin Zhang. 2024. Tyche: an efficient and general prefetcher for indirect memory accesses. *ACM Transactions on Architecture and Code Optimization* 21, 2 (2024), 1–26.
[52] Feng Xue, Junliang Wu, Chenji Han, Xinyu Li, Tingting Zhang, Tianyi Liu, and Fuxin Zhang. 2025. Augur: Semantics-Aware Temporal Prefetching for Linked Data Structure. *ACM Transactions on Architecture and Code Optimization* 22, 3

(2025), 1–27.
[53] Adi Yoaz, Mattan Erez, Ronny Ronen, and Stephan Jourdan. 1999. Speculation techniques for improving load related instruction scheduling. In *Proceedings of the 26th annual international symposium on Computer architecture*. 42–53.
[54] Siavash Zangeneh, Stephen Pruett, Sangkug Lym, and Yale N Patt. 2020. Branchnet: A convolutional neural network to predict hard-to-predict branches. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 118–130.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009